



**Master en Systèmes d'Informations et Données**

**Ecole Supérieure Polytech Diamniadio**

**Université Amadou Mahtar MBOW**

## **> Projet de Fin de Module : Virtualisation - Conteneurisation**



# **Mise en place d'un cluster Kubernetes avec 02 nœuds de contrôle**

Présenté par :

- **M Maguette DIOP**
- **M Mouhamet THIOUNE**

Superviseur : **Dr Massamba LO**

## PLAN

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>Introduction.....</b>                                                       | <b>3</b>  |
| <b>1- Présentation.....</b>                                                    | <b>3</b>  |
| <b>2- Architecture.....</b>                                                    | <b>4</b>  |
| <b>3- Préparation de l'environnement.....</b>                                  | <b>4</b>  |
| <b>4- Installation et configuration des composants.....</b>                    | <b>5</b>  |
| Désactivation du swap et configuration réseau.....                             | 6         |
| Installation du runtime containerd.....                                        | 6         |
| Installation de runc et des plugins CNI.....                                   | 6         |
| Installation des composants Kubernetes.....                                    | 7         |
| <b>5- Déploiement, configuration et intégration du cluster Kubernetes.....</b> | <b>8</b>  |
| Mise en place du Load Balancer.....                                            | 8         |
| Initialisation du control-plane (master1).....                                 | 8         |
| Déploiement du réseau de pods.....                                             | 9         |
| Ajout des autres nœuds.....                                                    | 10        |
| <b>6- Validation de la haute disponibilité.....</b>                            | <b>11</b> |
| Déploiement d'une application de démo.....                                     | 11        |
| Simulation de panne d'un master.....                                           | 12        |
| <b>Conclusion.....</b>                                                         | <b>12</b> |

# Introduction

Ce rapport expose de manière détaillée l'ensemble du processus de déploiement d'un cluster Kubernetes hautement disponible. L'enjeu principal réside dans la mise en place d'une architecture résiliente, tolérante aux pannes et capable d'assurer la continuité des services même en cas de défaillance partielle des nœuds constitutifs. La démarche suivie inclut la préparation des environnements systèmes, l'installation des composants nécessaires, l'initialisation du cluster avec haute disponibilité, l'ajout des nœuds de contrôle et de calcul, ainsi que la validation des fonctionnalités par le déploiement d'application de démonstration.

## 1- Présentation

Le choix d'une architecture Kubernetes hautement disponible répond à un impératif de fiabilité pour les environnements de production. L'indisponibilité d'un nœud de contrôle ou de travail ne doit pas entraîner l'interruption des services applicatifs. Sur AWS, cette exigence peut être satisfaite par une répartition judicieuse des ressources sur plusieurs zones de disponibilité, couplée à un mécanisme de répartition de charge.

Les principaux objectifs fixés sont les suivants :

- ➔ Assurer une redondance du control-plane grâce à 02 nœuds maîtres.
- ➔ Distribuer les charges de travail sur 2 nœuds de calcul.
- ➔ Mettre en œuvre un mécanisme de load balancing pour l'accès à l'apiserver de Kubernetes.
- ➔ Démontrer la résilience du cluster par des tests de panne et de reprise.

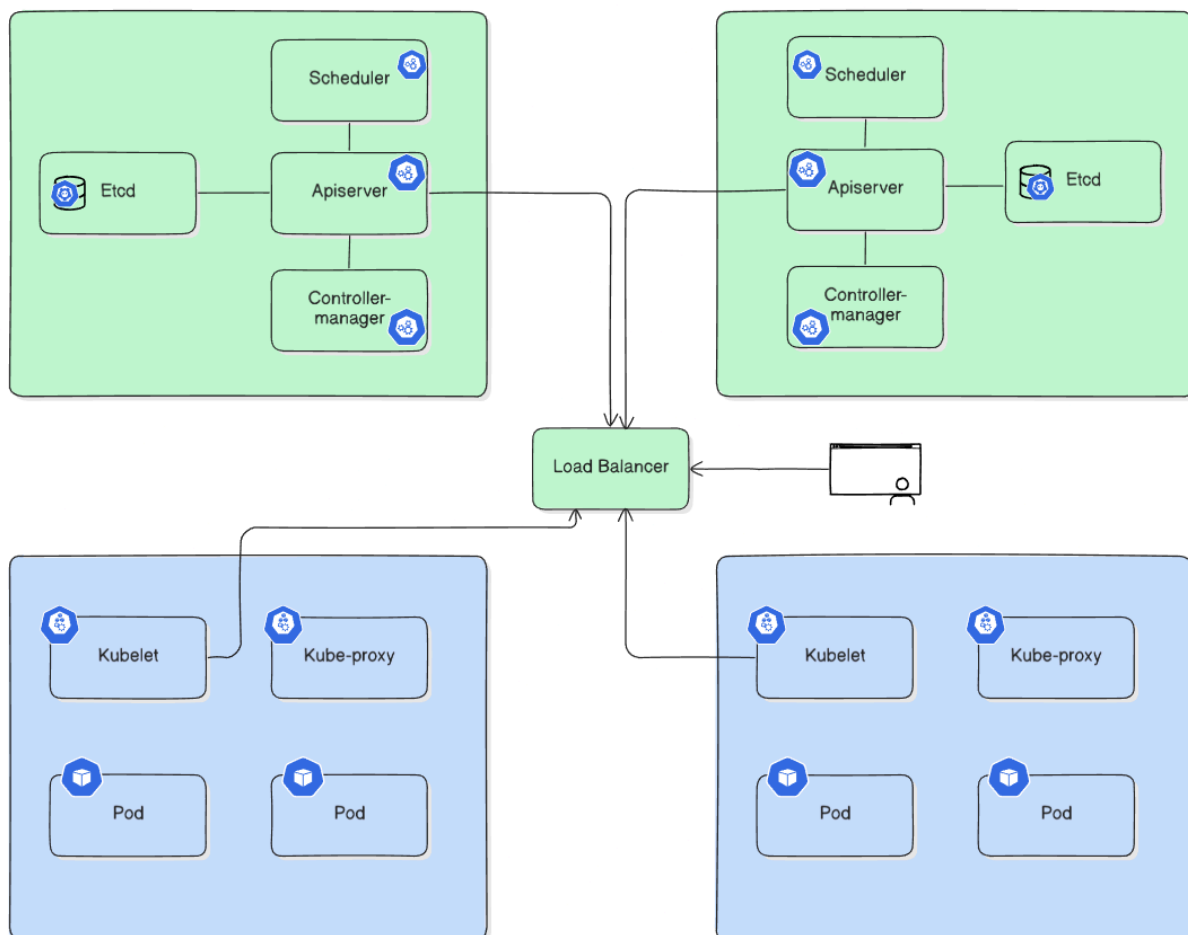
La mise en œuvre d'un cluster Kubernetes hautement disponible répond à l'objectif central de continuité de service. Dans un environnement distribué, la perte d'un nœud ou d'un composant ne doit pas conduire à une interruption. Kubernetes, par la redondance de son plan de contrôle et l'ordonnancement dynamique des charges de travail, fournit un cadre robuste et extensible. L'utilisation d'AWS permet de bénéficier d'une infrastructure flexible, adaptée aux contraintes de tolérance aux pannes et de montée en charge.

## 2- Architecture

L'architecture conçue repose sur une topologie hybride alliant équilibre des rôles et simplicité opérationnelle :

- **Deux nœuds control-plane (masters)** : hébergent l'**API Server**, le **scheduler**, le **controller-manager** et la base de données **etcd**.
- **Deux nœuds de travail (workers)** : destinés à l'hébergement des applications déployées sous forme de **pods**.
- **Un nœud Load Balancer (utilisant HAProxy)** : point d'entrée unique vers l'api Kubernetes, distribuant les requêtes vers les nœuds maîtres disponibles.

Cette architecture garantit la continuité de service même en cas de panne d'un des masters ou d'un worker.



### 3- Préparation de l'environnement

Le déploiement a été réalisé sur le cloud AWS. La mise en œuvre du cluster nécessite donc au préalable la création et la configuration des ressources. Cependant, il est important de mentionner que l'objectif général est d'avoir **05 hôtes disponibles et en réseau**. D'autres méthodes peuvent être utilisées telles que des machines physiques ou des machines virtuelles à travers un hyperviseur.

- 1. Virtual Private Cloud (VPC)

Un VPC spécifique est défini afin d'isoler le cluster. Deux sous-réseaux publics sont créés pour répartir les instances dans des zones de disponibilité distinctes, améliorant ainsi la tolérance aux pannes.

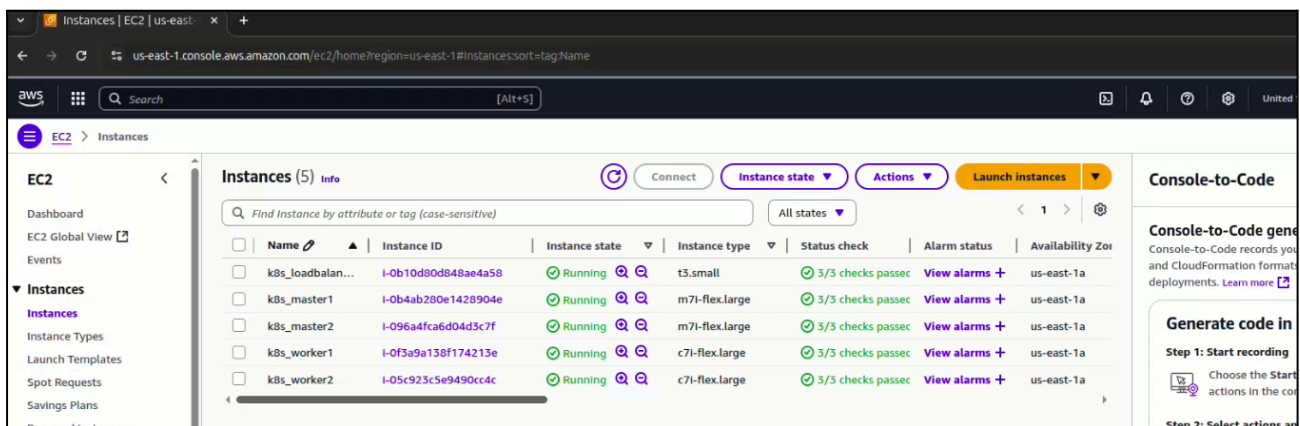
- 2. Security Groups (SG)

Les règles de pare-feu autorisant l'accès aux ports nécessaires :

- Port **6443** : apiserver
- Ports **10259** : scheduler
- Ports **10250** : kubelet
- Ports **10257** : controller-manager
- Ports **2379–2380** : etcd
- Ports **22** : pour accès ssh

- 3. Instances EC2

Des instances sont créées pour chaque rôle (masters, workers, load balancer). Les tailles de machines sont choisies selon les besoins recommandés pour un cluster Kubernetes (2vCPU + 4Go mémoire pour les masters nodes et 1vCPU + 2Go pour les workers nodes). Les adresses IP privées servent pour la communication interne.



## 4- Installation et configuration des composants

Avant toute initialisation, les systèmes hôtes doivent être correctement configurés pour supporter Kubernetes et containerd. Plusieurs opérations préliminaires sont réalisées : désactivation du swap, configuration des modules noyau, paramètres sysctl, et installation des outils réseau.

### Désactivation du swap et configuration réseau

```
root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~# swapoff -a
root@k8s-master2:~# sed -i 's/ swap / s/^#/' /etc/fstab
root@k8s-master2:~# cat <<EOF | tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
root@k8s-master2:~# modprobe overlay
root@k8s-master2:~# modprobe br_netfilter
root@k8s-master2:~# cat <<EOF | tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
root@k8s-master2:~# sysctl --system
* Applying /usr/lib/sysctl.d/10-apparmor.conf ...
* Applying /etc/sysctl.d/10-bufferbloat.conf ...
* Applying /etc/sysctl.d/10-console-messages.conf ...
* Applying /etc/sysctl.d/10-ipv6-privacy.conf ...
* Applying /etc/sysctl.d/10-kernel-hardening.conf ...
* Applying /etc/sysctl.d/10-magic-sysrq.conf ...
* Applying /etc/sysctl.d/10-map-count.conf ...
* Applying /etc/sysctl.d/10-network-security.conf ...
* Applying /etc/sysctl.d/10-ptrace.conf ...
```

La désactivation du swap est essentielle, Kubernetes ne tolérant pas son usage par le kubelet. Les modules `overlay` et `br_netfilter` sont activés pour permettre le trafic inter-pods et la communication au niveau des bridges. Les paramètres `sysctl` sont ajustés pour activer le forwarding IP et le filtrage réseau.

### Installation du runtime containerd

Le runtime containerd est installé manuellement, configuré avec `SystemdCgroup = true` afin d'assurer la compatibilité avec kubelet. Un service systemd est créé et activé.

```
root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~# curl -LO https://raw.githubusercontent.com/containerd/containerd/main/containerd.service
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 1248 100 1248    0     0  44411      0 --:--:-- --:--:-- --:--:-- 46222
root@k8s-master2:~#
root@k8s-master2:~# mv containerd.service /etc/systemd/system/
root@k8s-master2:~#
root@k8s-master2:~# systemctl daemon-reload
root@k8s-master2:~# systemctl enable --now containerd
root@k8s-master2:~# systemctl status containerd
● containerd.service - containerd container runtime
   Loaded: loaded (/etc/systemd/system/containerd.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-09-27 21:17:46 UTC; 56min ago
     Docs: https://containerd.io
   Main PID: 8706 (containerd)
    Tasks: 7
   Memory: 14.4M (peak: 15.1M)
      CPU: 16.818s
   CGroup: /system.slice/containerd.service
           └─8706 /usr/local/bin/containerd

Sep 27 21:17:46 k8s-master2 containerd[8706]: time="2025-09-27T21:17:46.820375235Z" level=info msg="Start subscribing >
Sep 27 21:17:46 k8s-master2 containerd[8706]: time="2025-09-27T21:17:46.820413410Z" level=info msg="Start recovering s>
Sep 27 21:17:46 k8s-master2 containerd[8706]: time="2025-09-27T21:17:46.820485584Z" level=info msg="serving... address=>
Sep 27 21:17:46 k8s-master2 containerd[8706]: time="2025-09-27T21:17:46.820518342Z" level=info msg="serving... address=>
Sep 27 21:17:46 k8s-master2 containerd[8706]: time="2025-09-27T21:17:46.820531002Z" level=info msg="Start event monito>
Sep 27 21:17:46 k8s-master2 containerd[8706]: time="2025-09-27T21:17:46.820545531Z" level=info msg="Start snapshots sy>
```

## Installation de runc et des plugins CNI

- **'runc'** est installé comme exécuteur bas niveau des conteneurs.

```
root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~# curl -LO https://github.com/opencontainers/runc/releases/download/v1.1.12/runc.amd64
sudo install -m 755 runc.amd64 /usr/local/sbin/runc
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 10.2M 100 10.2M    0     0  75.0M      0  --:--:-- --:--:-- --:--:--  75.0M
root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~#
```

- Les **plugins CNI** (Container Network Interface) sont installés dans **'/opt/cni/bin'**. Ils permettent à Kubernetes de gérer le réseau des pods.

```
root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~# curl -LO https://github.com/containernetworking/plugins/releases/download/v1.5.0/cni-plugins-linux-amd64-v1.5.0.tgz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  --:--:-- --:--:-- --:--:--    0
100 45.9M 100 45.9M    0     0 137M      0  --:--:-- --:--:-- --:--:-- 137M
root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~# mkdir -p /opt/cni/bin
root@k8s-master2:~#
root@k8s-master2:~# tar Cxvzf /opt/cni/bin cni-plugins-linux-amd64-v1.5.0.tgz
./
./dhcp
./loopback
```

## Installation des composants Kubernetes

Après ajout du dépôt officiel, **'kubelet'**, **'kubeadm'** et **'kubectl'** sont installés sur les nœuds master et worker. Ces paquets sont ensuite marqués en hold pour éviter les mises à jour intempestives.

```
root@k8s-master2:~#
root@k8s-master2:~# mkdir -p /etc/apt/keyrings
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.33/deb/Release.key | gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
root@k8s-master2:~# echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.33/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.33/deb/ /
root@k8s-master2:~# apt-get update
Hit:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Get:5 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.33/deb InRelease [1186 B]
Get:6 https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.33/deb Packages [8840 B]
Fetched 10.0 kB in 0s (35.0 kB/s)
Reading package lists... Done
root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~# apt-get install -y kubelet=1.33.0-1.1 kubeadm=1.33.0-1.1 kubectl=1.33.0-1.1 --allow-downgrades --allow-change-held-packages
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

## 5- Déploiement, configuration et intégration du cluster Kubernetes

### Mise en place du Load Balancer

Le serveur HAProxy est installé et configuré afin de relayer les requêtes sur le port **6443** vers les masters disponibles. Cette configuration permet d'utiliser un seul point d'entrée stable pour l'initialisation et l'administration du cluster. (Fichier de configuration `/etc/haproxy/haproxy.cfg`)

```
defaults
  log          global
  mode         http
  option       httplog
  option       dontlognull
  timeout connect 5000
  timeout client 50000
  timeout server 50000
  errorfile 400 /etc/haproxy/errors/400.http
  errorfile 403 /etc/haproxy/errors/403.http
  errorfile 408 /etc/haproxy/errors/408.http
  errorfile 500 /etc/haproxy/errors/500.http
  errorfile 502 /etc/haproxy/errors/502.http
  errorfile 503 /etc/haproxy/errors/503.http
  errorfile 504 /etc/haproxy/errors/504.http

frontend kubernetes-frontend
  bind *:6443
  default_backend kubernetes-backend

backend kubernetes-backend
  balance roundrobin
  server master1 <MASTER1_PRIVATE_IP>:6443 check
  server master2 <MASTER2_PRIVATE_IP>:6443 check
```

### Initialisation du control-plane (master1)

L'initialisation est effectuée avec la commande `'kubeadm init'`, spécifiant :

- l'adresse IP du loadbalancer comme point d'accès,
- le CIDR réseau des pods (`'192.168.0.0/16'`),
- l'adresse du master *leader* pour l'API Server.

```
kubeadm init --control-plane-endpoint "LOAD_BALANCER_PRIVATE_IP:6443" --upload-certs  
--pod-network-cidr=192.168.0.0/16 --apiserver-advertise-address=10.0.11.82
```

Le résultat fournit les commandes `'kubeadm join'` nécessaires pour l'intégration des autres nœuds.



```
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rotatable kubelet client certificate and key
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy
```

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Alternatively, if you are the root user, you can run:

```
export KUBECONFIG=/etc/kubernetes/admin.conf
```

You should now deploy a pod network to the cluster.

Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:  
<https://kubernetes.io/docs/concepts/cluster-administration/addons/>

You can now join any number of control-plane nodes running the following command on each as root:

```
kubeadm join 10.0.4.152:6443 --token uda23j.pjnvq2y38 \
--discovery-token-ca-cert-hash sha256:19347cdaad0ee00f4491c4d0660285160f99 \
--control-plane --certificate-key 37f140923ef902d699c790bb506418b09f48514c
```

Please note that the certificate-key gives access to cluster sensitive data, keep it secret!

As a safeguard, uploaded-certs will be deleted in two hours; If necessary, you can use  
"kubeadm init phase upload-certs --upload-certs" to reload certs afterward.

Then you can join any number of worker nodes by running the following on each as root:

```
kubeadm join 10.0.4.152:6443 --token uda23j.pjnvq2y38 \
--discovery-token-ca-cert-hash sha256:19347cdaad0ee00f4491c4d0660285160f99
```

## Déploiement du réseau de pods

Le réseau Calico est appliqué via le manifeste officiel. Celui-ci configure un overlay réseau assurant la communication entre les pods, quel que soit leur hôte.

L'initialisation du plan de contrôle est effectuée sur le premier nœud maître, avec un endpoint commun correspondant à l'adresse IP privée du **loadbalancer**:

Le réseau Calico est appliqué via le manifeste officiel. Celui-ci configure un overlay réseau assurant la communication entre les pods, quel que soit leur hôte.

```
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml
```

```
root@k8s-master1:~#
root@k8s-master1:~#
root@k8s-master1:~#
root@k8s-master1:~# kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.28.0/manifests/custom-resources.yaml
installation.operator.tigera.io/default created
apiserver.operator.tigera.io/default created
root@k8s-master1:~#
```

## Ajout des autres nœuds

Les autres nœuds maîtres rejoignent le cluster via la commande **join** fournie lors de l'initialisation :

```
kubeadm join 10.0.4.152:6443 --token uda23j.pjnvq2y38i00ryvz --discovery-token-ca-cert-hash sha256:19347cdaad0ee00f4491c4d0660285160f99122979a897685e71623034639c3e --control-plane --certificate-key 37f140923ef902d699c790bb506418b09f48514c86cc431238918344dd2f7...
```

```
root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~# kubeadm join 10.0.4.152:6443 --token uda23j.pjnvq2y38i00ryvz \
--discovery-token-ca-cert-hash sha256:19347cdaad0ee00f4491c4d0660285160f99122979a897685e71623034639c3e \
--control-plane --certificate-key 37f140923ef902d699c790bb506418b09f48514c86cc431238918344dd2f7...
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config --config your-config-file' to re-upload it.
[preflight] Running pre-flight checks before initializing the new control plane instance
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action beforehand using 'kubeadm config images pull'
W0927 22:21:05.643544 9943 checks.go:846] detected that the sandbox image "registry.k8s.io/pause:3.8" of the container runtime is inconsistent with that used by kubeadm. It is recommended to use "registry.k8s.io/pause:3.10" as the CRI sandbox image.
[download-certs] Downloading the certificates in Secret "kubeadm-certs" in the "kube-system" Namespace
[download-certs] Saving the certificates to the folder: "/etc/kubernetes/pki"
[certs] Using certificateDir folder "/etc/kubernetes/pki"
```

Une fois terminé, deux nœuds masters fonctionnels forment le cluster.

```
root@k8s-master1:~#
root@k8s-master1:~#
root@k8s-master1:~#
root@k8s-master1:~# kubectl get nodes
NAME STATUS ROLES AGE VERSION
k8s-master1 Ready control-plane 55m v1.33.0
k8s-master2 Ready control-plane 3m51s v1.33.0
root@k8s-master1:~# kubectl get pods -A
NAMESPACE NAME READY STATUS RESTARTS AGE
kube-system calico-kube-controllers-7498b9bb4c-hzv27 1/1 Running 0 22m
kube-system calico-node-ckz47 0/1 Running 0 3m59s
kube-system calico-node-kcmz4 0/1 Running 0 22m
kube-system coredns-674b8bbfcf-mjkhc 1/1 Running 0 55m
kube-system coredns-674b8bbfcf-mq9fj 1/1 Running 0 55m
kube-system etcd-k8s-master1 1/1 Running 0 55m
kube-system etcd-k8s-master2 1/1 Running 0 3m59s
kube-system kube-apiserver-k8s-master1 1/1 Running 0 55m
kube-system kube-apiserver-k8s-master2 1/1 Running 0 3m59s
kube-system kube-controller-manager-k8s-master1 1/1 Running 0 55m
kube-system kube-controller-manager-k8s-master2 1/1 Running 0 3m59s
kube-system kube-proxy-4km52 1/1 Running 0 3m59s
kube-system kube-proxy-wbjwc 1/1 Running 0 55m
kube-system kube-scheduler-k8s-master1 1/1 Running 0 55m
kube-system kube-scheduler-k8s-master2 1/1 Running 0 3m59s
root@k8s-master1:~#
```

Les nœuds workers rejoignent le cluster de la même manière, **sans l'option `--control-plane`** :

```
kubeadm join 10.0.4.152:6443 --token uda23j.pjnvq2y38i00ryvz --discovery-token-ca-cert-hash sha256:19347cdaad0ee00f4491c4d0660285160f99122979a897685e71623034639...
```

```
root@k8s-worker1:~#
root@k8s-worker1:~#
root@k8s-worker1:~#
root@k8s-worker1:~# kubectl join 10.0.4.152:6443 --token uda23j.pjnvq2y \
--discovery-token-ca-cert-hash sha256:19347cdaad0ee00f4491c4d0660285160f991
[preflight] Running pre-flight checks
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config --config your-config-file' to re-upload it.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 501.46878ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiserver and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

root@k8s-worker1:~#
```

Une fois l'opération terminée, l'ensemble des nœuds apparaît avec le statut **'Ready'**.

```
root@k8s-master1:~#
root@k8s-master1:~# kubectl get nodes

NAME           STATUS    ROLES    AGE   VERSION
k8s-master1    Ready     control-plane  67m   v1.33.0
k8s-master2    Ready     control-plane  15m   v1.33.0
k8s-worker1    Ready     <none>      5m22s v1.33.0
k8sworker2     Ready     <none>      17s   v1.33.0

root@k8s-master1:~#
```

## 6- Validation de la haute disponibilité

Afin de vérifier la stabilité et la résilience du cluster, plusieurs tests fonctionnels et de panne sont menés.

### Déploiement d'une application de démonstration

Un déploiement avec 3 réplicats est créé à partir d'une image nginx.

```
root@k8s-master1:~#
root@k8s-master1:~# kubectl create deployment webserver --image=nginx --replicas=3
deployment.apps/webserver created
root@k8s-master1:~#
root@k8s-master1:~#
root@k8s-master1:~#
root@k8s-master1:~# kubectl get deployment
NAME      READY   UP-TO-DATE   AVAILABLE   AGE
webserver 3/3      3            3           60s
root@k8s-master1:~# kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
webserver-765f5759d4-bg6t5         1/1     Running   0           84s
webserver-765f5759d4-sqhqx         1/1     Running   0           84s
webserver-765f5759d4-w4gsq         1/1     Running   0           84s
root@k8s-master1:~# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE           NOMINATED NODE   READINESS GATES
webserver-765f5759d4-bg6t5         1/1     Running   0           96s   192.168.194.67 k8s-worker1    <none>            <none>
webserver-765f5759d4-sqhqx         1/1     Running   0           96s   192.168.194.66 k8s-worker1    <none>            <none>
webserver-765f5759d4-w4gsq         1/1     Running   0           96s   192.168.8.2    k8sworker2     <none>            <none>
```

## Simulation de panne d'un master

Un des masters nodes est volontairement arrêté. Le cluster continue de fonctionner normalement grâce au load balancer qui redirige vers l'autre master.

```
root@k8s-master1:~#
Sep 27 23:49:11 k8s-master1 systemd[1]: kubelet.service
: Consumed 2min 246ms CPU time.
root@k8s-master1:~#
root@k8s-master1:~#
root@k8s-master1:~#
root@k8s-master1:~# sudo systemctl stop kubelet

root@k8s-master1:~# kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-master1   NotReady  control-plane 140m   v1.33.0
k8s-master2   Ready     control-plane 88m    v1.33.0
k8s-worker1   Ready     <none>      78m    v1.33.0
k8sworker2    Ready     <none>      72m    v1.33.0

root@k8s-master1:~# kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-master1   NotReady  control-plane 140m   v1.33.0
k8s-master2   Ready     control-plane 88m    v1.33.0
k8s-worker1   Ready     <none>      78m    v1.33.0
k8sworker2    Ready     <none>      73m    v1.33.0

root@k8s-master1:~#

root@k8s-master2:~#
root@k8s-master2:~#
root@k8s-master2:~# kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
k8s-master1   NotReady  control-plane 144m   v1.33.0
k8s-master2   Ready     control-plane 92m    v1.33.0
k8s-worker1   Ready     <none>      81m    v1.33.0
k8sworker2    Ready     <none>      76m    v1.33.0

root@k8s-master2:~#
root@k8s-master2:~# kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP             NODE
webserver-765f5759d4-bg6t5         1/1     Running   0           24m   192.168.194.67 k8s-worker1
webserver-765f5759d4-sqhqx         1/1     Running   0           24m   192.168.194.66 k8s-worker1
webserver-765f5759d4-w4gsq         1/1     Running   0           24m   192.168.8.2    k8sworker2
root@k8s-master2:~#
```

Les pods demeurent disponibles et joignables. Le control-plane reste fonctionnel. L'expérience met en évidence la robustesse de l'infrastructure et confirme l'efficacité des mécanismes de haute disponibilité mis en place.

## Conclusion

Le déploiement d'un cluster Kubernetes hautement disponible a été mené avec succès. La redondance du control-plane démontre la résilience de l'infrastructure. L'arrêt d'un nœud maître a validé la continuité de service, ce qui serait impossible s'il y avait un seul control-plane.

Cependant, l'architecture actuelle comporte encore des limites. Le load balancer représente un point de défaillance unique qu'il conviendrait de renforcer par une configuration en haute disponibilité, par exemple à travers service managé comme **AWS ELB** (Elastic Load Balancer). De plus, la persistance des données applicatives reste un défi majeur, car etcd et les volumes persistants nécessitent des stratégies de sauvegarde et de réplication robustes.

## Lien utiles

- [1] runc in Kubernetes <https://zesty.co/finops-glossary/runc-in-kubernetes/>
- [2] HAProxy on Ubuntu <https://www.haproxy.com/blog/how-to-install-haproxy-on-ubuntu>
- [3] CNI plugins <https://github.com/containernetworking/plugins>